

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

# ICT Class 9 - Formative Assessment 2

Formative Assessment (FA-2) - Answer Key

Class: Class 9

Assessment Type: Formative Assessment (FA-2)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Data Structures - Lists, Tuples, Dictionaries, Sets

## Answer Key - Grading Guidelines

**Note to Teacher:** This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

### Part A: Written Concept Paper (20 Marks)

#### Section I: MCQ (5 x 1 = 5 marks)

- Answer:** b) 3
  - Explanation:** Sets automatically remove duplicates. {1, 2, 3, 2, 1} becomes {1, 2, 3}, so `len(s)` returns 3.
- Answer:** c) `items()`
  - Explanation:** `dict.items()` returns a view object displaying a list of key-value tuples. `keys()` returns only keys, `values()` returns only values.
- Answer:** b) [2, 4, 6]
  - Explanation:** The list comprehension iterates through each row and each element, filtering even numbers (`x % 2 == 0`), producing [2, 4, 6].
- Answer:** d) Both B and C
  - Explanation:** Tuples can contain mixed data types and are generally faster than lists due to immutability (fixed memory).
- Answer:** a) {'a': 0, 'b': 0, 'c': 0}
  - Explanation:** `dict.fromkeys()` creates a dictionary with the given keys and the same default value for all keys.

#### Section II: Short Answer (5 x 2 = 10 marks)

- Differentiate between a list and a tuple.**
  - Answer:** Lists are mutable (can be modified after creation) while tuples are immutable (cannot be modified). Lists use square brackets [], tuples use parentheses ().
  - Marking:** 1 mark for mutability difference, 1 mark for two real-world use cases where tuples are preferred (e.g., coordinates, RGB colours, function return values).
- Write the output of the dictionary update code.**
  - Answer:** Output: {'x': 1, 'y': 3, 'z': 4}
  - Explanation:** `update()` merges d2 into d1. Since "y" exists in both, the value from d2 (3) overwrites the value in d1 (2).
  - Marking:** 1 mark for correct output, 1 mark for explanation of overwrite behaviour.
- Explain the difference between `remove()`, `pop()`, and `del`.**
  - Answer:**
    - `remove(x)`: Removes the first occurrence of x from the list. Returns nothing. Raises `ValueError` if not found.
    - `pop(index)`: Removes and returns the element at the given index. Default is the last element.
    - `del`: A keyword that removes an element by index or deletes the entire variable. E.g., `del lst[0]`.
  - Marking:** 1 mark for any two correct explanations with examples, 1 mark for the third.

4. Write a Python program to find intersection and union of two sets.

• Answer:

```
def intersection(set1, set2):
    result = set()
    for item in set1:
        if item in set2:
            result.add(item)
    return result

def union(set1, set2):
    result = set1.copy()
    for item in set2:
        if item not in result:
            result.add(item)
    return result

a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}

print("Intersection:", intersection(a, b)) # {4, 5}
print("Union:", union(a, b)) # {1, 2, 3, 4, 5, 6, 7, 8}
```

- **Marking:** 1 mark for intersection logic, 1 mark for union logic without using & or | operators.

5. What is list comprehension? Write one to generate Pythagorean triplets.

- **Answer:** List comprehension is a concise way to create lists using a single line of code with the syntax [expression for item in iterable if condition].

- **Marking:** 1 mark for definition, 1 mark for correct list comprehension:

```
triplets = [(a, b, c) for a in range(1, 21) for b in range(a, 21) for c in range(b, 21) if a**2 + b**2 == c**2]
print(triplets)
# Output: [(3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15)]
```

### Section III: Long Answer (5 x 1 = 5 marks)

1. Write a Python program implementing a word frequency counter.

• Answer:

```
import string

def word_frequency_counter(text):
    """Count word frequency in a paragraph."""
    # Remove punctuation and convert to lowercase
    cleaned = text.translate(str.maketrans('', '', string.punctuation)).lower()
    words = cleaned.split()

    # Count frequencies
    freq = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1

    # Sort by frequency descending
    sorted_words = sorted(freq.items(), key=lambda x: x[1], reverse=True)

    # Display top 5
    print("Top 5 most frequent words:")
```

```

for word, count in sorted_words[:5]:
    print(f" {word}: {count}")

# Words appearing exactly once
print("\nWords appearing exactly once:")
for word, count in sorted_words:
    if count == 1:
        print(f" {word}")

# Test
text = """Python is great and Python is easy. Python is used for data analysis.
Data analysis is fun. Analysis of data requires Python."""
word_frequency_counter(text)

```

- **Marking:** 1 mark for reading and cleaning text, 1 mark for counting frequencies, 1 mark for sorting and displaying top 5, 1 mark for finding unique words, 1 mark for code structure and test case.

## Part B: Hands-on Practical Lab Task (5 marks)

- **Answer:**

```

students = [
    {"name": "Amit", "roll_no": 1, "marks": 85},
    {"name": "Priya", "roll_no": 2, "marks": 92},
    {"name": "Rahul", "roll_no": 3, "marks": 78},
    {"name": "Sneha", "roll_no": 4, "marks": 88},
    {"name": "Vikram", "roll_no": 5, "marks": 65},
    {"name": "Ananya", "roll_no": 6, "marks": 95},
    {"name": "Deepak", "roll_no": 7, "marks": 72},
    {"name": "Meera", "roll_no": 8, "marks": 81},
    {"name": "Arjun", "roll_no": 9, "marks": 59},
    {"name": "Kavya", "roll_no": 10, "marks": 88},
    {"name": "Rohan", "roll_no": 11, "marks": 76},
    {"name": "Divya", "roll_no": 12, "marks": 91},
    {"name": "Suresh", "roll_no": 13, "marks": 68},
    {"name": "Pooja", "roll_no": 14, "marks": 83},
    {"name": "Manish", "roll_no": 15, "marks": 70},
]

def find_topper(students):
    return max(students, key=lambda s: s["marks"])

def class_average(students):
    return sum(s["marks"] for s in students) / len(students)

def below_average(students, avg):
    return [s for s in students if s["marks"] < avg]

def sort_by_marks(students):
    return sorted(students, key=lambda s: s["marks"], reverse=True)

def display_table(students):
    print(f"{'Name':<12} {'Roll No':<10} {'Marks':<8}")
    print("-" * 32)
    for s in students:
        print(f"{s['name']:<12} {s['roll_no']:<10} {s['marks']:<8}")

```

```

# Results
topper = find_topper(students)
avg = class_average(students)
below_avg = below_average(students, avg)
sorted_students = sort_by_marks(students)

print("=== Class Topper ===")
print(f"{topper['name']} - {topper['marks']} marks\n")

print(f"=== Class Average: {avg:.2f} ===\n")

print("=== Below Average Students ===")
display_table(below_avg)

print("\n=== Students Ranked by Marks ===")
display_table(sorted_students)

```

- **Marking:** 1 mark for creating student records, 1 mark for topper function, 1 mark for average and below-average functions, 1 mark for sorting, 1 mark for formatted display.

## Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)